

CODE_GUIDE.md — คู่มือได้ฉบับสมบูรณ์

สำหรับคนที่ไม่ถนัดโค้ด: ไฟล์นี้รวมโค้ดจริงทุกบรรทัดของทุกไฟล์ในโปรเจกต์ พร้อมคำอธิบายภาษาไทย ไล่ตามลำดับการทำงานของระบบ

สารบัญ

- [โครงสร้างโปรเจกต์](#)
- [จุดเริ่มต้น — index.html + main.tsx + App.tsx](#)
- [ประเภทข้อมูล — types.ts](#)
- [การ Auth — supabase.ts + AuthContext.tsx](#)
- [หน้าเข้าสู่ระบบ — Login.tsx + Consent.tsx](#)
- [แบบสอบถาม — Questionnaire.tsx](#)
- [Engine คำนวณ — metrics, riskCoefficient, riskCards](#)
- [การจัดสรรพอร์ต — capitalMarketAssumptions, broadAllocation, optimizer](#)
- [ภาษี — taxModule.ts](#)
- [หน้าผลลัพธ์ — Output1, Output2, Output3](#)
- [ฐานข้อมูล — dbTypes, index, localStorageProvider, supabaseProvider](#)
- [Global State — appStore.tsx](#)
- [Component ย่อย — MetricCard, RiskCard, Disclaimer](#)
- [Export — exportCSV, exportPDF](#)
- [ประวัติ — History.tsx](#)
- [Admin — Admin, AdminUser, AdminAssessment](#)
- [ข้อมูล Mock — dataLayer.ts](#)
- [Config — package.json, vite.config, tsconfig, index.css](#)

1. โครงสร้างโปรเจกต์

```
finance/
├─ index.html           ← หน้า HTML หลัก
├─ package.json         ← dependencies
├─ vite.config.ts       ← config build tool
├─ tsconfig.json        ← config TypeScript
├─ src/
│   ├─ main.tsx         ← จุดเริ่มต้น React
│   ├─ App.tsx          ← routing ทั้งหมด
│   └─ index.css        ← CSS global
```

└─ vite-env.d.ts	← type สำหรับ env vars
└─ engine/	← ตรรกะคำนวณ (ไม่มี UI)
└─ types.ts	← interface ทุกตัว
└─ metrics.ts	← คำนวณตัวชี้วัดทางการเงิน
└─ riskCards.ts	← การ์ดความเสี่ยง 10 ด้าน
└─ riskCoefficient.ts	← คะแนนและระดับความเสี่ยง
└─ capitalMarketAssumptions.ts	← ค่าสมมติตลาด
└─ broadAllocation.ts	← จัดสรรพอร์ต (4 สินทรัพย์)
└─ optimizer.ts	← Markowitz efficient frontier
└─ taxModule.ts	← ภาษีการลงทุน
└─ dataLayer.ts	← ข้อมูล mock หุ่น/ราคา
└─ lib/	
└─ supabase.ts	← Supabase client
└─ auth/	
└─ AuthContext.tsx	← Auth provider
└─ store/	
└─ appStore.tsx	← Global state (Context+Reducer)
└─ db/	
└─ dbTypes.ts	← Interface ฐานข้อมูล
└─ index.ts	← เลือก provider อัดโนมิติ
└─ localStorageProvider.ts	← บันทึกใน browser
└─ supabaseProvider.ts	← บันทึกใน Supabase
└─ components/	
└─ MetricCard.tsx	← การ์ดแสดงตัวเลข
└─ RiskCard.tsx	← การ์ดความเสี่ยง
└─ Disclaimer.tsx	← ข้อความแจ้งเตือน
└─ utils/	
└─ exportCSV.ts	← ส่งออกไฟล์ CSV
└─ exportPDF.ts	← ส่งออกไฟล์ PDF
└─ pages/	
└─ Login.tsx	← หน้าเข้าสู่ระบบ
└─ Consent.tsx	← ยินยอม PDPA
└─ Questionnaire.tsx	← แบบสอบถาม 11 ชั้น
└─ Output1.tsx	← ผล: ตัวชี้วัด + การ์ดเสี่ยง
└─ Output2.tsx	← ผล: พอร์ตพาย + breakdown
└─ Output3.tsx	← ผล: Efficient Frontier + ภาษี
└─ History.tsx	← ประวัติการประเมิน
└─ Admin.tsx	← หน้า admin รายชื่อลูกค้า
└─ AdminUser.tsx	← ประวัติของลูกค้าคนหนึ่ง
└─ AdminAssessment.tsx	← รายงานการประเมินเต็ม (admin)

2. จุดเริ่มต้น

index.html

```

<!DOCTYPE html>
<html lang="th">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>ที่ปรึกษาการเงิน</title>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>

```

```
<script type="module" src="/src/main.tsx"></script>
</body>
</html>
```

อธิบาย: ไฟล์ HTML เริ่มต้น ตั้งภาษาเป็น `th` (ไทย) มี `<div id="root">` ที่ React จะ inject UI เข้าไป และโหลด `main.tsx` เป็น module

`src/main.tsx`

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

อธิบาย: จุดเริ่มต้น React — หา element `#root` แล้ว render `<App>` ใส่ไป `StrictMode` ช่วยตรวจจับ bug ระหว่าง development

`src/App.tsx`

```
import React from 'react'
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { AuthProvider } from './auth/AuthContext'
import { AppProvider } from './store/appStore'
import Login from './pages/Login'
import Consent from './pages/Consent'
import Questionnaire from './pages/Questionnaire'
import Output1 from './pages/Output1'
import Output2 from './pages/Output2'
import Output3 from './pages/Output3'
import History from './pages/History'
import Admin from './pages/Admin'
import AdminUser from './pages/AdminUser'
import AdminAssessment from './pages/AdminAssessment'

export default function App() {
  return (
    <AuthProvider>
      <AppProvider>
        <BrowserRouter>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/consent" element={<Consent />} />
            <Route path="/" element={<Questionnaire />} />
            <Route path="/output1" element={<Output1 />} />
            <Route path="/output2" element={<Output2 />} />
          </Routes>
        </BrowserRouter>
      </AppProvider>
    </AuthProvider>
  )
}
```

```

    <Route path="/output3" element={<Output3 />} />
    <Route path="/history" element={<History />} />
    <Route path="/admin" element={<Admin />} />
    <Route path="/admin/:userId" element={<AdminUser />} />
    <Route path="/admin/:userId/:assessmentId" element={<AdminAssessment />} />
    <Route path="*" element={<Navigate to="/" />} />
  </Routes>
</BrowserRouter>
</AuthProvider>
</AuthProvider>
)
}

```

อธิบาย: ไฟล์หลักที่ตั้ง routing ทั้งหมด ห่อด้วย:

- `AuthProvider` — จัดการ login/logout
- `AppProvider` — เก็บผลการคำนวณระหว่าง session
- `BrowserRouter` — ใช้ URL จริง (ไม่ใช่ hash)

เส้นทาง	หน้า
<code>/login</code>	เข้าสู่ระบบ
<code>/consent</code>	ยินยอม PDPA
<code>/</code>	แบบสอบถาม
<code>/output1</code>	ผลตัวชี้วัด + การ์ดเสี่ยง
<code>/output2</code>	พอร์ดพาย
<code>/output3</code>	Efficient Frontier + ภาณี
<code>/history</code>	ประวัติการประเมิน
<code>/admin</code>	Admin: รายชื่อลูกค้า
<code>/admin/:userId</code>	Admin: ประวัติลูกค้า
<code>/admin/:userId/:assessmentId</code>	Admin: รายงานเต็ม

3. Types

`src/engine/types.ts`

```

export interface Answers {
  // — ส่วนบุคคล —
  fullName: string
  age: number
  gender: 'ชาย' | 'หญิง' | 'อื่นๆ'
  maritalStatus: 'โสด' | 'แต่งงาน' | 'หย่าร้าง' | 'หม้าย'
}

```

```
children: number
dependents: number
province: string
retirementAge: number
education: string
occupation: string
jobStability: 'สูง' | 'ปานกลาง' | 'ต่ำ'
hasChronicDisease: 'มี' | 'ไม่มี'

// — รายได้ —
mainIncome: number
annualBonus: number
sideIncome: number
passiveIncome: number
incomeConsistency: 'สม่ำเสมอมาก' | 'ค่อนข้างสม่ำเสมอ' | 'ผันผวน' | 'ผันผวนสูง'
emergencyMonthsHeld: number
layoffRisk: 'ต่ำ' | 'ปานกลาง' | 'สูง'

// — ค่าใช้จ่าย —
monthlyExpenses: number
monthlySavings: number
cashFlowProblems: 'ไม่เคย' | 'บางครั้ง' | 'บ่อย'
creditCardOverspend: 'ไม่เคย' | 'บางครั้ง' | 'บ่อย'

// — สินทรัพย์ —
cash: number
bankDeposit: number
thaiStocksAsset: number
foreignStocksAsset: number
crypto: number
goldAsset: number
realEstate: number

// —หนี้สิน—
mortgageDebt: number
carDebt: number
creditCardDebt: number
personalDebt: number
totalMonthlyDebtPayment: number
everDefaulted: 'ไม่เคย' | 'เคย'

// — ประกัน —
hasLifeInsurance: 'มี' | 'ไม่มี'
hasHealthInsurance: 'มี' | 'ไม่มี'
hasCriticalIllnessInsurance: 'มี' | 'ไม่มี'

// — ประสบการณ์ลงทุน —
hasInvestedBefore: 'เคย' | 'ไม่เคย'
portfolioDrop20Response: 'ซื้อเพิ่ม' | 'ถือไว้' | 'ลดบางส่วน' | 'ขายทั้งหมด'
mainDecisionFactor: 'วิเคราะห์เอง' | 'คำแนะนำผู้เชี่ยวชาญ' | 'แนวโน้มตลาดและข่าว' | 'กระแสโซเชียล'

// — ความรู้การลงทุน —
knowledgeAnswers: string[]

// — พฤติกรรม —
portfolioDrop1Month: 'ขายทั้งหมด' | 'ขายบางส่วน' | 'ถือไว้รอดู' | 'ซื้อเพิ่ม'
marketCrashMonths: 'หยุดลงทุน' | 'ลดการลงทุน' | 'ลงเท่าเดิม' | 'เพิ่มการลงทุน'
volatilityTolerance: 'ขาดทุนเล็กน้อยก็เครียด' | 'รับได้บ้าง' | 'รับได้ค่อนข้างมาก' | 'รับสูงได้'
investmentStyle: 'รักษาเงินต้น' | 'สมดุลเสี่ยง-ผลตอบแทน' | 'ต้องการผลตอบแทนสูง' | 'ยอมเสี่ยงสูงมาก'

// — เป้าหมายและภาษี —
```

```

    mainGoal: string
    targetReturnPercent: number
    investmentHorizonYears: number
    annualSalary: number
    investsSSF: 'ลงทุน' | 'ไม่ลงทุน'
    investsRMF: 'ลงทุน' | 'ไม่ลงทุน'

    // — เกษียณ —
    monthlyExpenseAfterRetirement: number
    currentRetirementSavings: number
    expectedLifespan: number
}

export interface MetricsResult {
    emergencyMonths: number | null
    dti: number | null
    savingsRate: number | null
    netWorth: number
    yearsToRetirement: number
    concentration: number | null
    knowledgeScore: number
    passiveRatio: number | null
    totalAssets: number
    totalLiabilities: number
    totalMonthlyIncome: number
    totalInvestmentAssets: number
    // normalized versions (0-100)
    emergencyMonthsN: number | null
    dtiN: number | null
    savingsRateN: number | null
    concentrationN: number | null
    passiveRatioN: number | null
    flags: string[]
}

export interface RiskCard {
    id: string
    name: string
    level: 'green' | 'yellow' | 'red'
    punchline: string
    advice: string
}

export interface RiskCoefficientResult {
    capacityScore: number
    toleranceScore: number
    finalScore: number
    riskLevel: 'conservative' | 'moderate' | 'aggressive'
    A: number
    reasons: string[]
}

export interface AssetWeights {
    thaiStocks: number
    foreignStocks: number
    bonds: number
    gold: number
    cash: number
}

export interface AllocationResult {

```

```

weights: AssetWeights
expectedReturn: number
expectedRisk: number
leverageFlag: boolean
yRisky: number
}

```

อธิบาย:

- `Answers` — คำตอบจากแบบสอบถาม 11 ขั้นตอน รวม ~50 ฟิลด์
- `MetricsResult` — ตัวชี้วัดที่คำนวณจาก Answers เช่น netWorth, DTI, knowledgeScore
- `RiskCard` — การวัดความเสี่ยงแต่ละด้าน (id, ชื่อ, ระดับสี, บทสรุป, คำแนะนำ)
- `RiskCoefficientResult` — ผลคะแนนความเสี่ยง พร้อม A (Risk Aversion) ช่วง 2–8
- `AllocationResult` — สัดส่วนพอร์ตที่แนะนำ 5 สินทรัพย์ + ผลตอบแทน/ความเสี่ยงคาดหวัง

4. Auth

`src/lib/supabase.ts`

```

import { createClient } from '@supabase/supabase-js'

const url = import.meta.env.VITE_SUPABASE_URL as string
const key = import.meta.env.VITE_SUPABASE_ANON_KEY as string

export const supabase = url && key ? createClient(url, key) : null

```

อธิบาย: สร้าง Supabase client จาก environment variables เท่านั้น (ไม่ hardcode) ถ้าไม่มี env vars จะ export `null` และระบบจะใช้ localStorage แทน

⚠️ **กฎด้านความปลอดภัย:** `VITE_SUPABASE_URL` และ `VITE_SUPABASE_ANON_KEY` ต้องเก็บใน `.env` เสมอ ห้าม hardcode ในโค้ด

`src/auth/AuthContext.tsx`

```

import React, { createContext, useContext, useEffect, useState } from 'react'
import type { User, Session } from '@supabase/supabase-js'
import { supabase } from '../lib/supabase'

interface AuthContextValue {
  user: User | null
  session: Session | null
  loading: boolean
  signUp: (email: string, password: string) => Promise<void>
  signIn: (email: string, password: string) => Promise<void>
}

```

```

    signInWithGoogle: () => Promise<void>
    signOut: () => Promise<void>
  }

  const AuthContext = createContext<AuthContextValue | null>(null)

  export function AuthProvider({ children }: { children: React.ReactNode }) {
    const [user, setUser] = useState<User | null>(null)
    const [session, setSession] = useState<Session | null>(null)
    const [loading, setLoading] = useState(true)

    useEffect(() => {
      if (!supabase) { setLoading(false); return }
      supabase.auth.getSession().then(({ data }) => {
        setSession(data.session)
        setUser(data.session?.user ?? null)
        setLoading(false)
      })
      const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => {
        setSession(session)
        setUser(session?.user ?? null)
      })
      return () => subscription.unsubscribe()
    }, [])

    async function signUp(email: string, password: string) {
      if (!supabase) return
      const { error } = await supabase.auth.signUp({ email, password })
      if (error) throw error
    }

    async function signIn(email: string, password: string) {
      if (!supabase) return
      const { error } = await supabase.auth.signInWithPassword({ email, password })
      if (error) throw error
    }

    async function signInWithGoogle() {
      if (!supabase) return
      const { error } = await supabase.auth.signInWithOAuth({
        provider: 'google',
        options: { redirectTo: window.location.origin + '/consent' },
      })
      if (error) throw error
    }

    async function signOut() {
      if (!supabase) return
      await supabase.auth.signOut()
    }

    return (
      <AuthContext.Provider value={{ user, session, loading, signUp, signIn, signInWithGoogle, signOut }}>
        {children}
      </AuthContext.Provider>
    )
  }

  export function useAuth() {
    const ctx = useContext(AuthContext)
    if (!ctx) throw new Error('useAuth must be inside AuthProvider')
  }

```



```
    return ctx
  }
}
```

อธิบาย:

- เมื่อโหลด ถึง session ที่มีอยู่แล้วจาก Supabase
- Subscribe ฟัง event เมื่อ login/logout เพื่ออัปเดต state
- `signInWithGoogle` redirect กลับมาที่ `/consent` หลัง OAuth สำเร็จ
- ถ้า `supabase = null` ทุก method ทำงานเงียบๆ (ไม่ error)

5. Login + Consent

`src/pages/Login.tsx` (สรุปหลัก)

หน้าเข้าสู่ระบบมี 2 mode: `signin` และ `signup`

ฟีเจอร์สำคัญ:

- ปุ่ม Google OAuth (`signInWithGoogle`)
- Email + Password (signin/signup)
- เมื่อ signup สำเร็จ แสดง banner ให้ verify email
- มีลิงก์ "ข้ามการเข้าสู่ระบบ" → `/` สำหรับใช้แบบไม่ login

`src/pages/Consent.tsx`

```
import React, { useState } from 'react'
import { useNavigate } from 'react-router-dom'

const CONSENT_KEY = 'pdpa_consent_given'

export function hasConsent(): boolean {
  return localStorage.getItem(CONSENT_KEY) === 'true'
}

export default function Consent() {
  const [checked, setChecked] = useState(false)
  const navigate = useNavigate()

  function handleAccept() {
    if (!checked) return
    localStorage.setItem(CONSENT_KEY, 'true')
    navigate('/')
  }

  function handleDecline() {
    navigate('/login')
  }
}
```

```

return (
  <div className="min-h-screen bg-gray-50 flex items-center justify-center px-4">
    <div className="bg-white rounded-xl shadow-sm p-8 max-w-lg w-full">
      <div className="text-center mb-6">
        <div className="text-3xl mb-2">🔒</div>
        <h1 className="text-xl font-bold text-gray-900">ขอความยินยอมเก็บข้อมูล</h1>
        <p className="text-sm text-gray-500 mt-1">ตาม พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล (PDPA) พ.ศ.
      </div>
      { /* ... เนื้อหา PDPA ... */ }
      <label className="flex items-start gap-3 mb-6 cursor-pointer">
        <input
          type="checkbox"
          checked={checked}
          onChange={e => setChecked(e.target.checked)}
          className="mt-0.5 w-4 h-4 text-indigo-600 rounded"
        />
        <span className="text-sm text-gray-700">
          ฉันอ่านและเข้าใจนโยบายข้างต้น และยินยอมให้เก็บข้อมูลการเงินส่วนบุคคลเพื่อวัตถุประสงค์ที่ระบุ
        </span>
      </label>
      <div className="flex gap-3">
        <button onClick={handleDecline} className="flex-1 py-2.5 border border-gray-300 text-gray-500">
          ไม่ยินยอม
        </button>
        <button onClick={handleAccept} disabled={!checked} className="flex-1 py-2.5 bg-indigo-600 text-white">
          ยินยอมและเริ่มใช้งาน
        </button>
      </div>
    </div>
  </div>
)
}

```

อธิบาย:

- เก็บความยินยอมใน `localStorage` key `pdpa_consent_given = 'true'`
- ปุ่ม "ยินยอม" จะ enable ก็ต่อเมื่อ tick checkbox แล้วเท่านั้น
- `hasConsent()` — function ที่ไฟล์อื่น import มาตรวจสอบ

6. Questionnaire

`src/pages/Questionnaire.tsx`

แบบสอบถาม 11 ขั้นตอน (TOTAL_STEPS = 11)

ขั้นตอน:

ขั้น	หัวข้อ	ข้อมูลที่เก็บ
1	ข้อมูลส่วนบุคคล	ชื่อ อายุ เพศ อาชีพ อายุเกษียณ
2	รายได้และความมั่นคง	รายได้ โบนัส passive income ความสม่ำเสมอ

ชั้น	หัวข้อ	ข้อมูลที่เก็บ
3	ค่าใช้จ่ายและกระแสเงินสด	ค่าใช้จ่าย เงินออม ปัญหากระแสเงินสด
4	สินทรัพย์	เงินสด หุ้น crypto ทอง อสังหา
5	หนี้สิน	บ้าน รถ บัตรเครดิต ค่างวด
6	ประกัน	ชีวิต สุขภาพ โรคร้ายแรง
7	ประสบการณ์ลงทุน	เคยลงทุนไหม ตอบสนองตลาดอย่างไร
8	แบบทดสอบความรู้	6 ข้อ (หุ้น/พันธบัตร/diversification/ETF/...)
9	พฤติกรรมการลงทุน	ปฏิกิริยาตลาดตก สไตล์การลงทุน
10	เป้าหมายและภาษี	เป้าหมาย ผลตอบแทนที่ต้องการ SSF/RMF
11	เกษียณ	ค่าใช้จ่ายหลังเกษียณ เงินที่มีแล้ว อายุขัย

คำถามความรู้ 6 ข้อ (พร้อมคำตอบที่ถูกต้อง):

```
const KNOWLEDGE_QUESTIONS = [
  { q: 'หุ้นกับพันธบัตรต่างกันอย่างไร?',
    // ✓ ถูก: 'หุ้นผันผวนและเสี่ยงสูงกว่า'
  },
  { q: 'การ diversification (กระจายการลงทุน) เพื่ออะไร?',
    // ✓ ถูก: 'ลดความเสี่ยงของพอร์ต'
  },
  { q: 'เมื่อดอกเบี้ยขึ้น สินทรัพย์ใดกระทบมากที่สุด?',
    // ✓ ถูก: 'พันธบัตร'
  },
  { q: 'การลงทุนระยะยาวช่วยอะไร?',
    // ✓ ถูก: 'ลดผลกระทบจากความผันผวนระยะสั้น'
  },
  { q: 'ETF คืออะไร?',
    // ✓ ถูก: 'กองทุนที่ซื้อขายในตลาดหุ้น'
  },
  { q: 'ถ้าไม่เข้าใจสินทรัพย์หนึ่ง ควรทำอะไร?',
    // ✓ ถูก: 'ศึกษาเพิ่มก่อนลงทุน'
  },
]
```

ค่า default เมื่อเริ่มแบบสอบถาม:

```
function emptyAnswers(): Partial<Answers> {
  return {
    fullName: '', age: 30, gender: 'ชาย', maritalStatus: 'โสด',
    children: 0, dependents: 0, province: '', retirementAge: 60,
    education: 'ตรี', occupation: 'พนักงานบริษัท', jobStability: 'ปานกลาง',
    hasChronicDisease: 'ไม่มี',
    mainIncome: 50000, annualBonus: 0, sideIncome: 0, passiveIncome: 0,
    incomeConsistency: 'ค่อนข้างสม่ำเสมอ', emergencyMonthsHeld: 3, layoffRisk: 'ปานกลาง',
    monthlyExpenses: 30000, monthlySavings: 10000,
    // ... (ทรัพย์สิน/หนี้สินทั้งหมด default = 0)
    portfolioDrop1Month: 'ถือไว้รอดู', marketCrashMonths: 'ลงเท่าเดิม',
    volatilityTolerance: 'รับได้บ้าง', investmentStyle: 'สมดุลเสี่ยง-ผลตอบแทน',
  }
}
```

```

    monthlyExpenseAfterRetirement: 30000, currentRetirementSavings: 0, expectedLifespan: 80,
  }
}

```

เมื่อกด "ดูผลการประเมิน" (ขั้น 11):

```

async function handleSubmit() {
  const answers = data as Answers
  const metrics = computeMetrics(answers) // คำนวณตัวชี้วัด
  const riskResult = computeRiskCoefficient(metrics, answers) // คะแนนความเสี่ยง
  const riskCards = assessAllRisks(metrics, answers) // การ์ดความเสี่ยง
  dispatch({ type: 'SET_ANSWERS', payload: answers })
  dispatch({ type: 'SET_METRICS', payload: metrics })
  dispatch({ type: 'SET_RISK_RESULT', payload: riskResult })
  dispatch({ type: 'SET_RISK_CARDS', payload: riskCards })
  db.saveCustomer({ /* บันทึกลง DB */ }).catch(console.error)
  navigate('/output1')
}

```

7. Engine

`src/engine/metrics.ts`

คำนวณตัวชี้วัดทางการเงินจาก Answers

```

import type { Answers, MetricsResult } from './types'

// คำตอบที่ถูกสำหรับ knowledge quiz
const CORRECT_ANSWERS = [
  'หุ้นผันผวนและเสี่ยงสูงกว่า',
  'ลดความเสี่ยงของพอร์ต',
  'พันธบัตร',
  'ลดผลกระทบจากความผันผวนระยะสั้น',
  'กองทุนที่ซื้อขายในตลาดหุ้น',
  'ศึกษาเพิ่มก่อนลงทุน',
]

function safeDiv(num: number, den: number): number | null {
  if (den === 0 || !isFinite(den)) return null
  return num / den
}

function normalize(value: number, min: number, max: number): number {
  if (max === min) return 50
  return Math.max(0, Math.min(100, ((value - min) / (max - min)) * 100))
}

export function computeMetrics(a: Answers): MetricsResult {
  const flags: string[] = []

  const totalMonthlyIncome =
    a.mainIncome + (a.annualBonus / 12) + a.sideIncome + a.passiveIncome

```

```

const totalAssets =
  a.cash + a.bankDeposit + a.thaiStocksAsset + a.foreignStocksAsset +
  a.crypto + a.goldAsset + a.realEstate

const totalLiabilities =
  a.mortgageDebt + a.carDebt + a.creditCardDebt + a.personalDebt

const totalInvestmentAssets =
  a.thaiStocksAsset + a.foreignStocksAsset + a.crypto + a.goldAsset

// เงินสำรองฉุกเฉิน = (เงินสด + ฝากธนาคาร) / ค่าใช้จ่ายต่อเดือน
const emergencyMonths = safeDiv(a.cash + a.bankDeposit, a.monthlyExpenses)

// DTI = ค่างวดต่อเดือน / รายได้รวม
const dti = safeDiv(a.totalMonthlyDebtPayment, totalMonthlyIncome)

// อัตราการออม = เงินออมต่อเดือน / รายได้รวม
const savingsRate = safeDiv(a.monthlySavings, totalMonthlyIncome)

const netWorth = totalAssets - totalLiabilities

const yearsToRetirement = Math.max(0, a.retirementAge - a.age)

// Concentration = สินทรัพย์ตัวใหญ่สุด / สินทรัพย์ลงทุนรวม
const maxSingleAsset = Math.max(
  a.thaiStocksAsset, a.foreignStocksAsset, a.crypto, a.goldAsset
)
const concentration = safeDiv(maxSingleAsset, totalInvestmentAssets)

// คะแนนความรู้ = จำนวนตอบถูก / 6
const correct = (a.knowledgeAnswers || []).filter(
  (ans, i) => ans === CORRECT_ANSWERS[i]
).length
const knowledgeScore = correct / 6

const passiveRatio = safeDiv(a.passiveIncome, totalMonthlyIncome)

return {
  emergencyMonths, dti, savingsRate, netWorth, yearsToRetirement,
  concentration, knowledgeScore, passiveRatio,
  totalAssets, totalLiabilities, totalMonthlyIncome, totalInvestmentAssets,
  emergencyMonthsN: emergencyMonths !== null ? normalize(emergencyMonths, 0, 12) : null,
  dtiN: dti !== null ? normalize(1 - dti, 0, 1) : null,
  savingsRateN: savingsRate !== null ? normalize(savingsRate, 0, 0.5) : null,
  concentrationN: concentration !== null ? normalize(1 - concentration, 0, 1) : null,
  passiveRatioN: passiveRatio !== null ? normalize(passiveRatio, 0, 1) : null,
  flags,
}
}

```

สูตรสำคัญ:

ตัวชี้วัด	สูตร
รายได้รวม/เดือน	mainIncome + annualBonus/12 + sideIncome + passiveIncome
ทรัพย์สินรวม	cash + bankDeposit + หุ้น + crypto + ทอง + อสังหา

ตัวชี้วัด	สูตร
หนี้สินรวม	บ้าน + รถ + บัตรเครดิต + ส่วนบุคคล
Net Worth	ทรัพย์สินรวม - หนี้สินรวม
Emergency Months	$(\text{cash} + \text{bankDeposit}) \div \text{monthlyExpenses}$
DTI	$\text{totalMonthlyDebtPayment} \div \text{รายได้รวม}$
Savings Rate	$\text{monthlySavings} \div \text{รายได้รวม}$
Concentration	สินทรัพย์ตัวใหญ่สุด $\div$ สินทรัพย์ลงทุนรวม
Knowledge Score	จำนวนตอบถูก $\div$ 6

`src/engine/riskCoefficient.ts`

คำนวณคะแนนความเสี่ยงและ Risk Aversion Coefficient A

```
import type { Answers, MetricsResult, RiskCoefficientResult } from '../types'

// น้ำหนักแต่ละด้านใน Capacity Score
const CAPACITY_WEIGHTS = {
  yearsToRetirement: 0.25, // เวลาเหลือก่อนเกษียณ
  incomeStability: 0.25, // ความมั่นคงของรายได้
  emergencyMonths: 0.20, // เงินสำรองฉุกเฉิน
  dti: 0.15, // ภาระหนี้
  dependents: 0.10, // จำนวนผู้พึ่งพิง
  healthAge: 0.05, // อายุ/สุขภาพ
}

const KNOWLEDGE_GATE_THRESHOLD = 0.5 // ต้องตอบถูกอย่างน้อย 3/6
const KNOWLEDGE_GATE_PENALTY = 15 // หักคะแนน 15 points

// แปลง job stability เป็นคะแนน 0-100
function incomeStabilityScore(a: Answers): number {
  const stabilityMap = { 'สูง': 80, 'ปานกลาง': 50, 'ต่ำ': 20 }
  const layoffMap = { 'ต่ำ': 80, 'ปานกลาง': 50, 'สูง': 20 }
  return (stabilityMap[a.jobStability] + layoffMap[a.layoffRisk]) / 2
}

// แปลง emergency months เป็นคะแนน
function emergencyScore(months: number | null): number {
  if (months === null) return 30
  if (months >= 12) return 100
  if (months >= 6) return 70
  if (months >= 3) return 40
  return 10
}

// แปลง DTI เป็นคะแนน
function dtiScore(dti: number | null): number {
  if (dti === null) return 50
  if (dti <= 0.15) return 100
  if (dti <= 0.30) return 70
}
```

```

    if (dti <= 0.40) return 40
    return 10
}

// จำนวนผู้พึ่งพิง
function dependentsScore(dependents: number): number {
    if (dependents === 0) return 100
    if (dependents === 1) return 70
    if (dependents <= 3) return 40
    return 20
}

// อายุ + โรคประจำตัว
function healthAgeScore(a: Answers): number {
    const ageScore = a.age < 30 ? 90 : a.age < 45 ? 70 : a.age < 55 ? 50 : 30
    const healthPenalty = a.hasChronicDisease === 'มี' ? 20 : 0
    return Math.max(0, ageScore - healthPenalty)
}

// ปีเหลือก่อนเกษียณ
function yearsToRetirementScore(years: number): number {
    if (years >= 30) return 100
    if (years >= 20) return 75
    if (years >= 10) return 50
    if (years >= 5) return 25
    return 10
}

// Tolerance Score: จากพฤติกรรมการลงทุน (subjective)
function computeToleranceScore(a: Answers): number {
    let score = 0

    // ปฏิกริยาพอร์ตลดลง 20% ใน 1 เดือน (น้ำหนัก 30%)
    const drop1mMap = {
        'ขายทั้งหมด': 10, 'ขายบางส่วน': 35, 'ถือไว้รอดู': 65, 'ซื้อเพิ่ม': 90,
    }
    score += (drop1mMap[a.portfolioDrop1Month] ?? 50) * 0.30

    // ปฏิกริยาตลาดตกต่อเนื่อง (น้ำหนัก 25%)
    const crashMap = {
        'หยุดลงทุน': 10, 'ลดการลงทุน': 35, 'ลงเท่าเดิม': 65, 'เพิ่มการลงทุน': 90,
    }
    score += (crashMap[a.marketCrashMonths] ?? 50) * 0.25

    // รับความผันผวนได้แค่ไหน (น้ำหนัก 25%)
    const volMap = {
        'ขาดทุนเล็กน้อยก็เครียด': 10, 'รับได้บ้าง': 40,
        'รับได้ค่อนข้างมาก': 70, 'รับสูงได้': 95,
    }
    score += (volMap[a.volatilityTolerance] ?? 50) * 0.25

    // สไตล์การลงทุน (น้ำหนัก 20%)
    const styleMap = {
        'รักษาเงินต้น': 10, 'สมดุลเสี่ยง-ผลตอบแทน': 45,
        'ต้องการผลตอบแทนสูง': 75, 'ยอมเสี่ยงสูงมาก': 95,
    }
    score += (styleMap[a.investmentStyle] ?? 50) * 0.20

    return Math.max(0, Math.min(100, score))
}

```

```

export function computeRiskCoefficient(
  metrics: MetricsResult,
  a: Answers
): RiskCoefficientResult {
  // คำนวณ Capacity Score (ความสามารถรับเสี่ยง)
  const ytr = yearsToRetirementScore(metrics.yearsToRetirement)
  const inc = incomeStabilityScore(a)
  const emg = emergencyScore(metrics.emergencyMonths)
  const dtiS = dtiScore(metrics.dti)
  const dep = dependentsScore(a.dependents)
  const hal = healthAgeScore(a)

  const capacityScore =
    ytr * CAPACITY_WEIGHTS.yearsToRetirement +
    inc * CAPACITY_WEIGHTS.incomeStability +
    emg * CAPACITY_WEIGHTS.emergencyMonths +
    dtiS * CAPACITY_WEIGHTS.dti +
    dep * CAPACITY_WEIGHTS.dependents +
    hal * CAPACITY_WEIGHTS.healthAge

  const toleranceScore = computeToleranceScore(a)

  // ใช้คะแนนที่ต่ำกว่า
  let finalScore = Math.min(capacityScore, toleranceScore)

  // Knowledge Gate: ถ้า tolerance > capacity แต่ความรู้ < 50% หัก 15 คะแนน
  if (toleranceScore > capacityScore && metrics.knowledgeScore < KNOWLEDGE_GATE_THRESHOLD) {
    finalScore = Math.max(0, finalScore - KNOWLEDGE_GATE_PENALTY)
  }

  // จัดระดับ
  const riskLevel =
    finalScore <= 33 ? 'conservative' :
    finalScore <= 66 ? 'moderate' : 'aggressive'

  // Risk Aversion Coefficient A (ช่วง 2-8)
  // A สูง = กลัวเสี่ยงมาก, A ต่ำ = ยอมเสี่ยงได้
  const A = Math.max(2, Math.min(8, 2 + (100 - finalScore) / 100 * 6))

  // สาเหตุ
  const reasons: string[] = []
  if (ytr < 50) reasons.push(`เหลือเวลาเกษียณ ${metrics.yearsToRetirement} ปี จำกัดการรับความเสี่ยงได้`)
  if (inc < 50) reasons.push('ความมั่นคงของรายได้ยังต่ำ')
  if (emg < 40) reasons.push('เงินสำรองฉุกเฉินยังไม่เพียงพอ')
  if (dtiS < 40) reasons.push('ภาระหนี้สูงเกินไป')
  if (toleranceScore < capacityScore) reasons.push('คุณรู้สึกไม่สบายใจกับความผันผวน จึงใช้คะแนนความเต็ม')
  if (metrics.knowledgeScore >= KNOWLEDGE_GATE_THRESHOLD) reasons.push('ความรู้การลงทุนอยู่ในระดับดี')

  return { capacityScore, toleranceScore, finalScore, riskLevel, A, reasons }
}

```

สูตร Risk Aversion A:

$$A = 2 + (100 - \text{finalScore}) / 100 \times 6$$

- finalScore = 100 → A = 2 (เสี่ยงสูงสุด)
- finalScore = 0 → A = 8 (ระวังสูงสุด)

ประเมินการ์ดความเสี่ยง 10 ด้าน เรียงจากด้านที่เสี่ยงที่สุดก่อน

```
import type { Answers, MetricsResult, RiskCard } from './types'

const T = {
  emergencyRed: 3,          // น้อยกว่า 3 เดือน = red
  emergencyYellow: 6,       // น้อยกว่า 6 เดือน = yellow
  dtiRed: 0.40,              // มากกว่า 40% = red
  dtiYellow: 0.30,          // มากกว่า 30% = yellow
  concentrationWarn: 0.50,  // กระจุกมากกว่า 50% = yellow
  savingsRateGood: 0.20,    // ออม 20%+ = good
  retirementGapFallback: 300, // fallback ถ้าไม่มี lifespan/retirementAge
}

export function assessAllRisks(metrics: MetricsResult, a: Answers): RiskCard[] {
  const cards: RiskCard[] = []

  // — การ์ดที่ 1: ความมั่นคงของอาชีพ —
  const incLevel =
    a.jobStability === 'สูง' ? 'green' :
    a.jobStability === 'ปานกลาง' ? 'yellow' : 'red'
  cards.push({ id: 'income_stability', name: 'ความมั่นคงของอาชีพ', level: incLevel,
    punchline: incLevel === 'green' ? 'อาชีพมั่นคง ความเสี่ยงด้านรายได้ต่ำ'
      : incLevel === 'yellow' ? 'ความมั่นคงอาชีพปานกลาง ควรสร้าง buffer'
      : 'ความมั่นคงต่ำ ควรสร้างเงินสำรองและรายได้เสริม',
    advice: incLevel !== 'green' ? 'พิจารณหารายได้เสริม และเพิ่มเงินสำรองฉุกเฉิน'
      : 'รักษาความมั่นคงนี้ไว้' })

  // — การ์ดที่ 2: ความสม่ำเสมอของรายได้ —
  const consLevel: RiskCard['level'] =
    a.incomeConsistency === 'สม่ำเสมอมาก' ? 'green' :
    (a.incomeConsistency === 'ค่อนข้างสม่ำเสมอ' || a.incomeConsistency === 'ผันผวน') ? 'yellow' :
  cards.push({ id: 'income_consistency', name: 'ความสม่ำเสมอของรายได้', level: consLevel,
    punchline: consLevel === 'green' ? 'รายได้สม่ำเสมอ วางแผนการเงินได้แม่นยำ'
      : consLevel === 'yellow' ? 'รายได้ค่อนข้างผันผวน ควรวางแผนรับมือ'
      : 'รายได้ผันผวนสูง ความเสี่ยงด้านกระแสเงินสดสูง',
    advice: consLevel !== 'green' ? 'สร้างเงินสำรองฉุกเฉินให้มากขึ้น และหารายได้ passive เพิ่ม'
      : 'ดีแล้ว รักษาให้คงเส้นคงวา' })

  // — การ์ดที่ 3: สภาพคล่อง (เงินสำรองฉุกเฉิน) —
  const em = metrics.emergencyMonths
  const emLevel: RiskCard['level'] =
    em === null ? 'red' :
    em < T.emergencyRed ? 'red' :
    em < T.emergencyYellow ? 'yellow' : 'green'
  cards.push({ id: 'liquidity', name: 'สภาพคล่องและเงินสำรอง', level: emLevel,
    punchline: em === null ? 'ไม่มีข้อมูลค่าใช้จ่าย ประเมินไม่ได้'
      : emLevel === 'green' ? `มีเงินสำรองฉุกเฉิน ${em.toFixed(1)} เดือน เพียงพอแล้ว`
      : emLevel === 'yellow' ? `มีเงินสำรองฉุกเฉินเพียง ${em.toFixed(1)} เดือน ควรเพิ่มเป็น 6 เดือน`
      : `เงินสำรองฉุกเฉินน้อยมาก (${em?.toFixed(1) ?? 0} เดือน) เสี่ยงสูงมาก`,
    advice: emLevel !== 'green' ? 'ควรสะสมเงินสำรองฉุกเฉินให้ได้ 6 เดือนก่อนลงทุน'
      : 'รักษาระดับนี้ไว้' })

  // — การ์ดที่ 4: ภาระหนี้สิน —
  const dti = metrics.dti
  const defaulted = a.everDefaulted === 'เคย'
```

```

const debtLevel: RiskCard['level'] =
  defaulted || (dti !== null && dti > T.dtiRed) ? 'red' :
  (dti !== null && dti > T.dtiYellow) ? 'yellow' : 'green'
cards.push({ id: 'debt', name: 'ภาระหนี้สิน', level: debtLevel,
  punchline: debtLevel === 'red'
    ? (defaulted ? 'เคยผิดนัดชำระหนี้ ความน่าเชื่อถือทางการเงินต่ำ'
      : `DTI สูงถึง ${((dti ?? 0) * 100).toFixed(1)}% ภาระหนักเกินไป`)
    : debtLevel === 'yellow' ? `DTI อยู่ที่ ${((dti ?? 0) * 100).toFixed(1)}% ควรระวัง`
    : `DTI อยู่ที่ ${dti !== null ? ((dti) * 100).toFixed(1) : 'N/A'}% อยู่ในเกณฑ์ดี`,
  advice: debtLevel !== 'green' ? 'วางแผนลดหนี้ก่อนเพิ่มการลงทุน'
    : 'ภาระหนี้ดีแล้ว' })

// — การ์ดที่ 5: ความเสี่ยงตลาด —
const totalInv = metrics.totalInvestmentAssets
const riskyRatio = totalInv > 0 ? (a.thaiStocksAsset + a.foreignStocksAsset + a.crypto) / to
const marketLevel: RiskCard['level'] =
  riskyRatio > 0.8 ? 'red' : riskyRatio > 0.6 ? 'yellow' : 'green'
cards.push({ id: 'market', name: 'ความเสี่ยงด้านตลาด', level: marketLevel,
  punchline: marketLevel === 'red' ? `สินทรัพย์เสี่ยงสูงถึง ${((riskyRatio * 100).toFixed(0))}% ของพอร์ต`
    : marketLevel === 'yellow' ? `สินทรัพย์เสี่ยง ${((riskyRatio * 100).toFixed(0))}% ควรพิจารณาระยะ`
    : 'สัดส่วนสินทรัพย์เสี่ยงอยู่ในระดับที่เหมาะสม',
  advice: marketLevel !== 'green' ? 'เพิ่มสัดส่วนตราสารหนี้หรือสินทรัพย์ปลอดภัย'
    : 'รักษาสัดส่วนนี้ไว้' })

// — การ์ดที่ 6: การกระจุกตัว หรือโอกาสเริ่มลงทุน —
const conc = metrics.concentration
const noInvestments = metrics.totalInvestmentAssets === 0
if (noInvestments) {
  cards.push({
    id: 'concentration', name: 'โอกาสในการลงทุน', level: 'green',
    punchline: 'คุณยังไม่มีการลงทุน ลองพิจารณาเริ่มลงทุนเพื่อให้เงินของคุณเติบโตในระยะยาว ทั้งนี้การลงทุนมีความเสี่ยง'
    advice: 'เมื่อพร้อม ควรเริ่มด้วยสินทรัพย์ที่เข้าใจและกระจายไปหลายประเภทตั้งแต่ต้น เพื่อลดความเสี่ยงจากตัวเดียว',
  })
} else {
  const concLevel: RiskCard['level'] =
    conc !== null && conc > T.concentrationWarn ? 'yellow' : 'green'
  cards.push({
    id: 'concentration', name: 'การกระจุกตัวของสินทรัพย์', level: concLevel,
    punchline: concLevel === 'yellow'
      ? `สินทรัพย์ลงทุนตัวเดียวมากถึง ${((conc! * 100).toFixed(1))}% ของพอร์ตลงทุน ถ้าตัวนั้นร่วงพอร์ตกระทบหนัก`
      : `พอร์ตมีการกระจายตัวที่ดี ตัวใหญ่สุดคิดเป็น ${((conc! * 100).toFixed(1))}%`,
    advice: concLevel === 'yellow'
      ? 'เพิ่มการกระจายไปยังสินทรัพย์ประเภทอื่น เพื่อลดความเสี่ยงจากตัวเดียว'
      : 'รักษการกระจายตัวนี้ไว้',
  })
}

// — การ์ดที่ 7: พฤติกรรมการลงทุน —
const behLevel: RiskCard['level'] =
  a.portfolioDrop1Month === 'ขายทั้งหมด' ? 'red' :
  a.portfolioDrop1Month === 'ขายบางส่วน' ? 'yellow' : 'green'
cards.push({ id: 'behavioral', name: 'พฤติกรรมการลงทุน', level: behLevel,
  punchline: behLevel === 'red' ? 'แนวโน้มขายตอนต้นตระหนก อาจทำให้ขาดทุนจริง'
    : behLevel === 'yellow' ? 'มีแนวโน้มขายบางส่วนเมื่อตลาดร่วง ระวัง panic sell'
    : 'พฤติกรรมการลงทุนดี มีวินัย ไม่ตื่นตระหนก',
  advice: behLevel !== 'green' ? 'ลงทุนในสิ่งที่เข้าใจ และกำหนด stop-loss ไว้ล่วงหน้า'
    : 'รักษาวินัยนี้ไว้' })

// — การ์ดที่ 8: ความพร้อมเกษียณ —
const monthlyRetirement = a.monthlyExpenseAfterRetirement || a.monthlyExpenses || 0
const lifeExp = a.expectedLifespan || 0

```

```

const retAge = a.retirementAge || 0
const retirementMonthsDerived = (lifeExp - retAge) * 12
const usingFallback = retirementMonthsDerived <= 0
const retirementMonths = usingFallback ? T.retirementGapFallback : retirementMonthsDerived
const retirementNeeded = monthlyRetirement * retirementMonths
const currentSavings = a.currentRetirementSavings || 0
const yearsLeft = metrics.yearsToRetirement
const projectedSavings = currentSavings + (a.monthlySavings || 0) * 12 * yearsLeft
const gap = retirementNeeded - projectedSavings
const retLevel: RiskCard['level'] = gap > retirementNeeded * 0.5 ? 'red' : gap > 0 ? 'yellow' : 'green'
const retirementYears = Math.round(retirementMonths / 12)
cards.push({
  id: 'retirement', name: 'ความพร้อมเกษียณ', level: retLevel,
  punchline: retLevel === 'green'
    ? `แนวโน้มเงินเกษียณอยู่ที่ ${fmtBaht(projectedSavings)} ครอบคลุมความต้องการ ${fmtBaht(retirementNeeded)} ปีหลังเกษียณ${usingFallback ? ' (หรือ ${fmtBaht(retirementNeeded)} ปีหากไม่มีเงินออม)'}` : 'ดีแล้ว! เงินออมเพียงพอแล้ว',
  advice: retLevel !== 'green'
    ? `เพิ่มการออมเพื่อเกษียณ เช่น RMF/SSF และลงทุนให้เงินงอกเงย เพื่อปิด gap ${fmtBaht(gap)} ` : 'เส้นทางเกษียณดูดีแล้ว รักษาระเบียบวินัยการออมไว้',
})

// — การ์ดที่ 9: สุขภาพและประกัน —
const noInsurance =
  a.hasLifeInsurance === 'ไม่มี' &&
  a.hasHealthInsurance === 'ไม่มี' &&
  a.hasCriticalIllnessInsurance === 'ไม่มี'
const partialInsurance =
  !noInsurance && (
    a.hasLifeInsurance === 'ไม่มี' ||
    a.hasHealthInsurance === 'ไม่มี'
  )
const insLevel: RiskCard['level'] = noInsurance ? 'red' : partialInsurance ? 'yellow' : 'green'
cards.push({ id: 'insurance', name: 'สุขภาพและประกัน', level: insLevel,
  punchline: insLevel === 'red' ? 'ไม่มีประกันเลย ความเสี่ยงด้านสุขภาพสูงมาก' :
    insLevel === 'yellow' ? 'มีประกันบางส่วน ควรเพิ่มความคุ้มครอง' :
    'มีประกันครบถ้วน ความเสี่ยงด้านสุขภาพต่ำ',
  advice: insLevel !== 'green' ? 'พิจารณาทำประกันสุขภาพและประกันชีวิตให้ครบก่อนลงทุน' :
    'ดีแล้ว รักษาความคุ้มครองนี้ไว้' })

// — การ์ดที่ 10: ประสิทธิภาพด้านภาษี —
const hasSSForRMF = a.investsSSF === 'ลงทุน' || a.investsRMF === 'ลงทุน'
const taxLevel: RiskCard['level'] = hasSSForRMF ? 'green' : 'yellow'
cards.push({ id: 'tax', name: 'ประสิทธิภาพด้านภาษี', level: taxLevel,
  punchline: hasSSForRMF ? 'ใช้ SSF/RMF ลดหย่อนภาษีได้ - ดี' :
    'ยังไม่ได้ใช้ SSF/RMF เพื่อลดหย่อนภาษี',
  advice: !hasSSForRMF ? 'พิจารณาลงทุน SSF (ลดหย่อนได้สูงสุด 200,000 บาท) หรือ RMF เพื่อลดภาษีและออมเงิน' :
    'ดีแล้ว ตรวจสอบให้แน่ใจว่าได้ลดหย่อนได้สูงสุด' })

// เรียงจากเสี่ยงที่สุด (red → yellow → green)
const order = { red: 0, yellow: 1, green: 2 }
return cards.sort((a, b) => order[a.level] - order[b.level])
}

```

8. Portfolio

src/engine/capitalMarketAssumptions.ts

ค่าสมมติตลาดทุน (ต้อง update ก่อน deploy จริง)

```
export const ASSET_NAMES = ['thaiStocks', 'foreignStocks', 'bonds', 'gold'] as const
export type AssetName = typeof ASSET_NAMES[number]

export const CMA = {
  assets: {
    // Sharpe ต่อสินทรัพย์ (vs Rf=2%): Thai≈0.23, Foreign≈0.30, Bonds≈0.10, Gold≈0.13
    // Tangency portfolio Sharpe ≈ 0.38-0.45 หลังกระจาย
    // y*(A=4) ≈ 0.75 → ~25% cash; y*(A=8) ≈ 0.38 → ~62% cash
    thaiStocks: { expectedReturn: 0.07, volatility: 0.22, label: 'หุ้นไทย' },
    foreignStocks: { expectedReturn: 0.08, volatility: 0.20, label: 'หุ้นต่างประเทศ' },
    bonds: { expectedReturn: 0.025, volatility: 0.05, label: 'ตราสารหนี้' },
    gold: { expectedReturn: 0.04, volatility: 0.16, label: 'ทองคำ' },
  },
  riskFreeRate: 0.02, // พันธบัตรรัฐ 2%
  cashLabel: 'เงินสด/พันธบัตรรัฐ',
  // Matrix สหสัมพันธ์ [thaiStocks, foreignStocks, bonds, gold]
  correlations: [
    [1.00, 0.60, -0.10, 0.10],
    [0.60, 1.00, -0.20, 0.05],
    [-0.10, -0.20, 1.00, 0.10],
    [0.10, 0.05, 0.10, 1.00],
  ],
}
```

ตารางสมมติฐาน:

สินทรัพย์	ผลตอบแทนคาดหวัง	ความผันผวน
หุ้นไทย	7%	22%
หุ้นต่างประเทศ	8%	20%
ตราสารหนี้	2.5%	5%
ทองคำ	4%	16%
เงินสด (Rf)	2%	0%

src/engine/broadAllocation.ts

จัดสรรพอร์ตด้วย Markowitz (4 สินทรัพย์ + cash) ใช้ใน Output2

```
import type { AllocationResult } from './types'
import { CMA, ASSET_NAMES } from './capitalMarketAssumptions'

// Matrix operations สำหรับ Markowitz
function buildCov(): number[][] {
  // สร้าง Covariance matrix จาก correlations × volatilities
  const n = ASSET_NAMES.length
```

```

const sigmas = ASSET_NAMES.map(k => CMA.assets[k].volatility)
return Array.from({ length: n }, (_, i) =>
  Array.from({ length: n }, (_, j) =>
    CMA.correlations[i][j] * sigmas[i] * sigmas[j]
  )
)
}

function invert4x4(m: number[][]): number[][] {
  // Gauss-Jordan elimination - invert 4x4 matrix
  const n = m.length
  const aug = m.map((row, i) => [...row, ...Array.from({ length: n }, (_, j) => i === j ? 1 : 0)])
  for (let col = 0; col < n; col++) {
    // หา pivot
    let pivot = col
    for (let row = col + 1; row < n; row++)
      if (Math.abs(aug[row][col]) > Math.abs(aug[pivot][col])) pivot = row
    [aug[col], aug[pivot]] = [aug[pivot], aug[col]]
    const div = aug[col][col]
    for (let j = 0; j < 2 * n; j++) aug[col][j] /= div
    for (let row = 0; row < n; row++) {
      if (row === col) continue
      const factor = aug[row][col]
      for (let j = 0; j < 2 * n; j++) aug[row][j] -= factor * aug[col][j]
    }
  }
  return aug.map(row => row.slice(n))
}

export function broadAllocation(A: number): AllocationResult {
  const Rf = CMA.riskFreeRate
  const mus = ASSET_NAMES.map(k => CMA.assets[k].expectedReturn)
  const excessReturns = mus.map(m => m - Rf)

  const cov = buildCov()
  const covInv = invert4x4(cov)

  // Tangency weights:  $w \propto \Sigma^{-1} \times (\mu - Rf)$ 
  const rawW = matVec(covInv, excessReturns)
  const sumW = rawW.reduce((s, v) => s + v, 0)
  const tangencyW = rawW.map(w => w / sumW)

  // Tangency portfolio stats
  const E_t = dot(tangencyW, mus)
  const sigma_t = Math.sqrt(/* portfolio variance */)

  // จุดที่เหมาะสมบน Capital Allocation Line
  //  $y^* = (E_t - Rf) / (A \times \sigma_t^2)$ 
  let yRisky = (E_t - Rf) / (A * sigma_t * sigma_t)
  let leverageFlag = false
  if (yRisky > 1) { yRisky = 1; leverageFlag = true } // clamp ไม่เกิน 100%
  if (yRisky < 0) yRisky = 0

  const ySafe = 1 - yRisky

  const weights = {
    thaiStocks: yRisky * tangencyW[0],
    foreignStocks: yRisky * tangencyW[1],
    bonds: yRisky * tangencyW[2],
    gold: yRisky * tangencyW[3],
    cash: ySafe,
  }
}

```

```

    }

    return {
      weights,
      expectedReturn: E_t * yRisky + Rf * ySafe,
      expectedRisk: sigma_t * yRisky,
      leverageFlag,
      yRisky,
    }
  }
}

```

สูตรหลัก:

- **Tangency Portfolio:** น้ำหนัก $\propto \Sigma^{-1} \times (\mu - R_f)$ — พอร์ตที่มี Sharpe ratio สูงสุด
- $y^* = (E_T - R_f) / (A \times \sigma_T^2)$ — สัดส่วนที่ลงในพอร์ตเสี่ยง
- เงินที่เหลือ $(1 - y^*)$ ไปอยู่ใน cash/พันธบัตรรัฐ

`src/engine/optimizer.ts`

Markowitz สำหรับ Output3 (ใช้กับหุ่นรายตัวและ animation)

```

export function efficientFrontier(
  returns: number[], cov: number[][], numPoints = 30
): PortfolioPoint[] {
  // Gradient descent หา minimum variance portfolio สำหรับ target return ต่างๆ
  // 2000 iterations per point
  const minR = Math.min(...returns)
  const maxR = Math.max(...returns)
  const points: PortfolioPoint[] = []
  for (let i = 0; i <= numPoints; i++) {
    const target = minR + (maxR - minR) * (i / numPoints)
    const w = minVarWeights(returns, cov, target, returns.length)
    // ...
    points.push({ sigma, mu, weights: w })
  }
  return points
}

export function tangencyPortfolio(
  returns: number[], cov: number[][], Rf: number
): TangencyResult {
  // Random restart 50 ครั้ง, gradient ascent บน Sharpe ratio
  // 1000 iterations ต่อ restart — เลือก Sharpe สูงสุด
  // ...
}

export function optimalForPerson(
  tangency: TangencyResult, Rf: number, A: number
): OptimalResult {
  //  $y^* = (E - R_f) / (A \times \sigma^2)$ 
  let yRisky = (tangency.E - Rf) / (A * tangency.sigma * tangency.sigma)
  // clamp [0, 1]
  // ...
}

```

ความต่างจาก broadAllocation:

- `broadAllocation` ใช้ matrix inversion แนนอน (เร็ว แนนอน)
- `optimizer` ใช้ numerical gradient (ยืดหยุ่นกว่า สำหรับ interactive chart)

9. Tax

`src/engine/taxModule.ts`

คำนวณภาษีการลงทุนตามกฎหมายไทย (อัปเดตตาม 2024-2025)

```
export const TAX_CONSTANTS = {
  progressiveRates: [
    { upTo: 150_000, rate: 0.00 }, // 0%
    { upTo: 300_000, rate: 0.05 }, // 5%
    { upTo: 500_000, rate: 0.10 }, // 10%
    { upTo: 750_000, rate: 0.15 }, // 15%
    { upTo: 1_000_000, rate: 0.20 }, // 20%
    { upTo: 2_000_000, rate: 0.25 }, // 25%
    { upTo: 5_000_000, rate: 0.30 }, // 30%
    { upTo: Infinity, rate: 0.35 }, // 35%
  ],
  dividendWithholdingThai: 0.10, // หัก ณ ที่จ่าย ปันผลหุ้นไทย 10%
  dividendWithholdingForeign: 0.15, // ปันผลหุ้นต่างประเทศ 15%
  ssfMaxDeduction: 200_000, // SSF หักได้สูงสุด 200,000 บาท
  ssfMaxPctOfIncome: 0.30, // SSF ≤ 30% ของรายได้
  rmfMaxPctOfIncome: 0.30, // RMF ≤ 30% ของรายได้
  rmfCombinedMax: 500_000, // RMF + SSF + อื่นๆ รวม ≤ 500,000
  foreignGainResidencyDays: 180, // ต้องอยู่ไทย > 180 วัน จึงต้องเสียภาษีกำไร
  foreignGainRuleYear: 2024, // กฎใหม่ปี 2024
  standardDeductionEmployed: 0.50, // หักค่าใช้จ่าย 50% ของรายได้
  standardDeductionMax: 100_000, // สูงสุด 100,000 บาท
  personalAllowance: 60_000, // ค่าลดหย่อนส่วนตัว 60,000 บาท
}

export function afterTaxReturn(
  portfolio: PortfolioTaxInput,
  person: PersonTaxInfo
): TaxResult {
  // 1. ภาษีปันผลหุ้นไทย = ปันผล × 10%
  // 2. ภาษีปันผลหุ้นต่างประเทศ = ปันผล × 15%
  // 3. กำไรหุ้นต่างประเทศ (ถ้าอยู่ไทย > 180 วัน และโอนกลับ) → ภาษีก้าวหน้า
  // 4. ประหยัดภาษีจาก SSF: คำนวณจาก progressive tax
  // 5. ประหยัดภาษีจาก RMF: คำนวณต่อจาก SSF
  // คืนค่า: grossReturn, afterTaxReturn, ssfTaxSaving, rmfTaxSaving
}
```

10. Output Pages

src/pages/Output1.tsx

แสดงผลตัวชี้วัดและการวัดความเสี่ยงทั้งหมด

สิ่งที่แสดง:

- MetricCards: Net Worth, เงินสำรองฉุกเฉิน, DTI, Savings Rate, รายได้, คะแนนความรู้
- RiskCards: การ์ดทั้งหมดที่ส่งคืนจาก `assessAllRisks` (เรียงแดงก่อน)
- กล้องระดับความเสี่ยงโดยรวม: conservative/moderate/aggressive พร้อม A value และเหตุผล
- ปุ่ม "ดูคำแนะนำพอร์ต" → `/output2`

src/pages/Output2.tsx

แสดงสัดส่วนพอร์ตที่แนะนำ

สิ่งที่แสดง:

- Donut pie chart (Recharts PieChart)
- Breakdown bars: หุ้นไทย, หุ้นต่างประเทศ, ตราสารหนี้, ทองคำ, เงินสด (สัดส่วน %)
- ผลตอบแทนคาดหวัง + ความเสี่ยง σ
- ปุ่ม "ดูรายละเอียดพอร์ต" → `/output3`

การคำนวณพอร์ต:

```
useEffect(() => {
  if (!state.allocation && state.riskResult) {
    const result = broadAllocation(state.riskResult.A)
    dispatch({ type: 'SET_ALLOCATION', payload: result })
  }
}, [])
```

คำนวณ `broadAllocation(A)` เพียงครั้งเดียวเมื่อยังไม่มีผล

src/pages/Output3.tsx

หน้าวิเคราะห์เชิงลึก

สิ่งที่แสดง:

1. **พอร์ตจริง** — สัดส่วนหลังปรับ A ($y_{\text{Risky}} \times \text{tangency weights} + y_{\text{Safe}}$ ใน cash)
2. **Tangency Portfolio** — พอร์ตเสี่ยงที่ Sharpe สูงสุด (เหมือนกันทุกคน)
3. **Efficient Frontier Chart** (Recharts LineChart):
 - เส้นสีเทา = Efficient Frontier
 - เส้นประสีน้ำเงิน = CAL (Capital Allocation Line)
 - เส้นประสีเหลือง = Indifference Curve
 - จุดสีเขียว T = Tangency

- จุดสี่ลัม = จุดของผู้ใช้
- Slider A ปรับได้ 2–8 แบบ real-time

4. Tax module — ผลตอบแทนก่อน/หลังภาษี (ฐาน 1 ล้านบาท)

5. Export CSV + PDF

Indifference Curve:

$$U = E - 0.5 \times A \times \sigma^2$$

เส้นที่ผู้ลงทุนรู้สึก "เท่ากัน" ทุกจุดบนเส้น

11. Database

src/db/dbTypes.ts

```
export interface CustomerRecord {
  id: string
  createdAt?: string
  userId?: string
  fullName: string
  age: number
  occupation?: string
  province?: string
  riskLevel: 'conservative' | 'moderate' | 'aggressive'
  riskA: number
  netWorth: number
  answers: Answers
  metrics: MetricsResult
  riskResult: RiskCoefficientResult
  riskCards: RiskCard[]
  allocation: AllocationResult | null
}

export interface DbProvider {
  saveCustomer(data: Omit<CustomerRecord, 'id' | 'createdAt' | 'userId'>): Promise<CustomerRecord>
  getCustomers(): Promise<CustomerRecord[]>
  getCustomersById(userId: string): Promise<CustomerRecord[]>
  deleteCustomer(id: string): Promise<void>
  updateCustomer(id: string, data: Partial<CustomerRecord>): Promise<CustomerRecord>
}
```

src/db/index.ts

```
import { supabase } from '../lib/supabase'
import { supabaseProvider } from './supabaseProvider'
import { localStorageProvider } from './localStorageProvider'
```

```
export const db = supabase ? supabaseProvider : localStorageProvider
```

อธิบาย: เลือก provider อัตโนมัติ — ถ้ามี Supabase ใช้ cloud, ถ้าไม่มีใช้ localStorage

src/db/localStorageProvider.ts

```
// Key ที่ใช้เก็บใน localStorage
const KEY = 'finance_customers'

export const localStorageProvider: DbProvider = {
  async saveCustomer(data) {
    const records = getAll()
    const record: CustomerRecord = {
      ...data,
      id: `local_${Date.now()}_${Math.random().toString(36).slice(2)}`,
      createdAt: new Date().toISOString(),
    }
    records.push(record)
    localStorage.setItem(KEY, JSON.stringify(records))
    return record
  },

  async getCustomers() {
    return getAll().reverse() // ล่าสุดก่อน
  },

  async getCustomersByUserId(userId) {
    return getAll().filter(r => r.userId === userId)
  },

  async deleteCustomer(id) {
    const filtered = getAll().filter(r => r.id !== id)
    localStorage.setItem(KEY, JSON.stringify(filtered))
  },

  async updateCustomer(id, data) {
    const records = getAll()
    const idx = records.findIndex(r => r.id === id)
    if (idx === -1) throw new Error('Not found')
    records[idx] = { ...records[idx], ...data }
    localStorage.setItem(KEY, JSON.stringify(records))
    return records[idx]
  },
}

function getAll(): CustomerRecord[] {
  try {
    return JSON.parse(localStorage.getItem(KEY) || '[]')
  } catch { return [] }
}
```

src/db/supabaseProvider.ts

```

// ตาราง: assessments
// คอลัมน์: user_id, full_name, age, occupation, province,
//         risk_level, risk_a, net_worth,
//         answers, metrics, risk_result, risk_cards, allocation (JSONB)

export const supabaseProvider: DbProvider = {
  async saveCustomer(data) {
    const { data: { user } } = await supabase!.auth.getUser()
    const { data: row, error } = await supabase!
      .from('assessments')
      .insert({
        user_id: user?.id,
        full_name: data.fullName,
        age: data.age,
        occupation: data.occupation,
        province: data.province,
        risk_level: data.riskLevel,
        risk_a: data.riskA,
        net_worth: data.netWorth,
        answers: data.answers,
        metrics: data.metrics,
        risk_result: data.riskResult,
        risk_cards: data.riskCards,
        allocation: data.allocation,
      })
      .select().single()
    if (error) throw error
    return mapRow(row)
  },

  async getCustomers() {
    const { data: { user } } = await supabase!.auth.getUser()
    const { data, error } = await supabase!
      .from('assessments')
      .select('*')
      .eq('user_id', user?.id)
      .order('created_at', { ascending: false })
    if (error) throw error
    return (data || []).map(mapRow)
  },
  // ...
}

function mapRow(row): CustomerRecord {
  return {
    id: row.id,
    createdAt: row.created_at,
    userId: row.user_id,
    fullName: row.full_name || '',
    age: row.age || 0,
    occupation: row.occupation || '',
    province: row.province || '',
    riskLevel: row.risk_level || 'moderate',
    riskA: row.risk_a || 0,
    netWorth: row.net_worth || 0,
    answers: row.answers,
    metrics: row.metrics,
    riskResult: row.risk_result,
    riskCards: row.risk_cards || [],
    allocation: row.allocation || null,
  }
}

```

```
}  
}
```

12. Store

src/store/appStore.tsx

```
import React, { createContext, useContext, useReducer } from 'react'  
import type { Answers, MetricsResult, RiskCard, RiskCoefficientResult, AllocationResult } from  
  
interface AppState {  
  answers: Answers | null  
  metrics: MetricsResult | null  
  riskResult: RiskCoefficientResult | null  
  riskCards: RiskCard[]  
  allocation: AllocationResult | null  
}  
  
type Action =  
  | { type: 'SET_ANSWERS'; payload: Answers }  
  | { type: 'SET_METRICS'; payload: MetricsResult }  
  | { type: 'SET_RISK_RESULT'; payload: RiskCoefficientResult }  
  | { type: 'SET_RISK_CARDS'; payload: RiskCard[] }  
  | { type: 'SET_ALLOCATION'; payload: AllocationResult }  
  
function reducer(state: AppState, action: Action): AppState {  
  switch (action.type) {  
    case 'SET_ANSWERS': return { ...state, answers: action.payload }  
    case 'SET_METRICS': return { ...state, metrics: action.payload }  
    case 'SET_RISK_RESULT': return { ...state, riskResult: action.payload }  
    case 'SET_RISK_CARDS': return { ...state, riskCards: action.payload }  
    case 'SET_ALLOCATION': return { ...state, allocation: action.payload }  
    default: return state  
  }  
}  
  
const initial: AppState = {  
  answers: null, metrics: null, riskResult: null, riskCards: [], allocation: null,  
}  
  
const AppContext = createContext<{ state: AppState; dispatch: React.Dispatch<Action> } | null>  
  
export function AppProvider({ children }: { children: React.ReactNode }) {  
  const [state, dispatch] = useReducer(reducer, initial)  
  return <AppContext.Provider value={{ state, dispatch }}>{children}</AppContext.Provider>  
}  
  
export function useApp() {  
  const ctx = useContext(AppContext)  
  if (!ctx) throw new Error('useApp must be inside AppProvider')  
  return ctx  
}
```

อธิบาย: Global state ระหว่างหน้า Output1 → Output2 → Output3 ใช้ React Context + useReducer ข้อมูลหายเมื่อ refresh หน้า (ไม่ได้ persist)

13. Components

src/components/MetricCard.tsx

```
interface MetricCardProps {
  label: string
  value: string
  sub?: string
  color?: 'default' | 'green' | 'yellow' | 'red'
}

export function MetricCard({ label, value, sub, color = 'default' }: MetricCardProps) {
  const colorMap = {
    default: 'border-gray-200 bg-white',
    green: 'border-green-200 bg-green-50',
    yellow: 'border-yellow-200 bg-yellow-50',
    red: 'border-red-200 bg-red-50',
  }
  return (
    <div className={`border rounded-xl p-4 ${colorMap[color]}`>
      <p className="text-xs text-gray-500 mb-1">{label}</p>
      <p className="text-lg font-bold text-gray-900">{value}</p>
      {sub && <p className="text-xs text-gray-400 mt-0.5">{sub}</p>}
    </div>
  )
}
```

src/components/RiskCard.tsx

```
interface RiskCardProps { card: RiskCard }

export function RiskCardComponent({ card }: RiskCardProps) {
  const colorMap = {
    green: { border: 'border-green-400', badge: 'bg-green-100 text-green-800', label: 'ดี' },
    yellow: { border: 'border-yellow-400', badge: 'bg-yellow-100 text-yellow-800', label: 'ควรระวัง' },
    red: { border: 'border-red-400', badge: 'bg-red-100 text-red-800', label: 'ควรรีบแก้ไข' },
  }
  const c = colorMap[card.level]
  return (
    <div className={`border-l-4 ${c.border} bg-white rounded-xl p-4 shadow-sm`}>
      <div className="flex items-start justify-between mb-2">
        <h3 className="font-semibold text-gray-800">{card.name}</h3>
        <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${c.badge}`}>{c.label}</span>
      </div>
      <p className="text-sm text-gray-700 mb-2">{card.punchline}</p>
      <p className="text-xs text-gray-500 italic">{card.advice}</p>
    </div>
  )
}
```

```
)  
}
```

src/components/Disclaimer.tsx

```
export function Disclaimer() {  
  return (  
    <p className="text-xs text-gray-400 text-center mt-4 px-4">  
      ระบบนี้เป็นเครื่องมือประเมินเบื้องต้นเพื่อการศึกษาเท่านั้น  
      ไม่ใช่คำแนะนำการลงทุนที่ได้รับอนุญาตจาก ก.ล.ด.  
      กรุณาปรึกษาที่ปรึกษาทางการเงินที่มีใบอนุญาตก่อนตัดสินใจลงทุน  
    </p>  
  )  
}
```

14. Export

src/utils/exportCSV.ts

```
export function exportMetricsCSV(  
  metrics: MetricsResult,  
  riskResult: RiskCoefficientResult,  
  riskCards: RiskCard[],  
  allocation: AllocationResult | null,  
  name = 'ผู้ใช้'  
) {  
  const lines: string[] = []  
  // Header  
  lines.push(row('ที่ปรึกษาการเงิน - ผลการประเมิน'))  
  lines.push(row('ชื่อ', name))  
  lines.push(row('วันที่ประเมิน', new Date().toLocaleDateString('th-TH')))  
  lines.push(row('Disclaimer', DISCLAIMER))  
  lines.push('')  
  
  // ตัวชี้วัด  
  lines.push(row('=== ตัวชี้วัดทางการเงิน ==='))  
  lines.push(row('ทรัพย์สินสุทธิ (Net Worth)', metrics.netWorth, 'บาท'))  
  // ...  
  
  // การ์ดความเสี่ยง  
  lines.push(row('=== การ์ดความเสี่ยง ==='))  
  riskCards.forEach(c => lines.push(row(c.name, c.level, c.punchline, c.advice)))  
  
  // พอร์ต  
  if (allocation) {  
    lines.push(row('=== สัดส่วนพอร์ตที่แนะนำ ==='))  
    lines.push(row('หุ้นไทย', (allocation.weights.thaiStocks * 100).toFixed(1)))  
    // ...  
  }  
}
```

```
// สร้างไฟล์และดาวน์โหลด
const csv = lines.join('\n')
const blob = new Blob(['' + csv], { type: 'text/csv;charset=utf-8;' })
// BOM ทำให้ Excel เปิดภาษาไทยได้ถูกต้อง
const url = URL.createObjectURL(blob)
const a = document.createElement('a')
a.href = url
a.download = `financial-assessment-${Date.now()}.csv`
a.click()
URL.revokeObjectURL(url)
}
```

src/utils/exportPDF.ts

```
import jsPDF from 'jspdf'
import html2canvas from 'html2canvas'

const DISCLAIMER = 'เอกสารนี้เป็นการประเมินเบื้องต้นเพื่อการศึกษาเท่านั้น ไม่ใช่คำแนะนำการลงทุนที่การันตีผล ' +
  'กรุณาปรึกษาผู้เชี่ยวชาญทางการเงินก่อนตัดสินใจลงทุน'

export async function exportToPDF(elementId: string, filename = 'financial-assessment.pdf') {
  const el = document.getElementById(elementId)
  if (!el) throw new Error(`Element #${elementId} not found`)

  // แปลง DOM element เป็น canvas (scale 2x = high DPI)
  const canvas = await html2canvas(el, {
    scale: 2, useCORS: true, backgroundColor: '#ffffff', scrollY: -window.scrollY,
  })

  const pdf = new jsPDF({ orientation: 'portrait', unit: 'mm', format: 'a4' })
  const pageW = pdf.internal.pageSize.getWidth()
  const pageH = pdf.internal.pageSize.getHeight()
  const margin = 10
  const contentW = pageW - margin * 2
  const imgH = (canvas.height * contentW) / canvas.width

  // หั่นภาพเป็น slice ตาม page height
  let y = margin
  let remainH = imgH
  while (remainH > 0) {
    const sliceH = Math.min(remainH, pageH - margin * 2)
    const srcY = (imgH - remainH) * (canvas.height / imgH)
    const srcH = sliceH * (canvas.height / imgH)
    const sliceCanvas = document.createElement('canvas')
    sliceCanvas.width = canvas.width
    sliceCanvas.height = srcH
    const ctx = sliceCanvas.getContext('2d')!
    ctx.drawImage(canvas, 0, srcY, canvas.width, srcH, 0, 0, canvas.width, srcH)
    pdf.addImage(sliceCanvas.toDataURL('image/png'), 'PNG', margin, y, contentW, sliceH)
    remainH -= sliceH
    if (remainH > 0) { pdf.addPage(); y = margin }
  }

  // Disclaimer + วันที่ footer
  pdf.setFontSize(7)
  pdf.setTextColor(150, 150, 150)
```

```
pdf.text(DISCLAIMER, margin, pageH - 6, { maxWidth: contentW })
pdf.setTextColor(180, 180, 180)
pdf.text(`สร้างเมื่อ: ${new Date().toLocaleDateString('th-TH', { dateStyle: 'full' })}`, margin,
  pdf.save(filename)
}
```

15. History

`src/pages/History.tsx`

หน้าประวัติการประเมินของผู้ใช้

ฟีเจอร์:

- โหลด records ทั้งหมดจาก `db.getCustomers()`
- แสดงรายการ: ชื่อ วันที่ ระดับเสี่ยง Net Worth
- ปุ่ม "โหลดผล" — restore ผลการคำนวณทั้งหมดกลับสู่ store แล้ว navigate ไป `/output1`
- ปุ่ม CSV/PDF export ต่อ record
- ปุ่ม ลบ (ขอ confirm ก่อน)

```
function loadRecord(r: CustomerRecord) {
  dispatch({ type: 'SET_ANSWERS', payload: r.answers })
  dispatch({ type: 'SET_METRICS', payload: r.metrics })
  dispatch({ type: 'SET_RISK_RESULT', payload: r.riskResult })
  dispatch({ type: 'SET_RISK_CARDS', payload: r.riskCards })
  if (r.allocation) dispatch({ type: 'SET_ALLOCATION', payload: r.allocation })
  navigate('/output1')
}
```

16. Admin

`src/pages/Admin.tsx`

```
const ADMIN_UID = import.meta.env.VITE_ADMIN_USER_ID as string
```

การทำงาน:

1. ตรวจสอบว่า `user.id === ADMIN_UID` — ถ้าไม่ใช่แสดง 403
2. ดึงลูกค้าทั้งหมดจาก Supabase (`assessments` table, ไม่ filter by `user_id`)
3. Group by `userId` เป็น `UserFolder`
4. Search by ชื่อ

5. Export CSV ทั้งหมด

6. คลิก Folder → `/admin/:userId`

`src/pages/AdminUser.tsx`

- รับ `:userId` จาก URL
- Query `assessments` filter by `user_id`
- แสดงรายการประวัติของลูกค้าคนนั้น
- คลิก → `/admin/:userId/:assessmentId`

`src/pages/AdminAssessment.tsx`

หน้ารายงานเต็มสำหรับ admin — แสดงทุกอย่างที่ผู้ใช้ใน Output1+2+3 พร้อมกัน:

- Metric Cards
- Risk Cards
- Overall Risk Level
- Pie Chart (พอร์ต)
- Tangency Portfolio
- Efficient Frontier Chart + Slider A
- Tax Module
- Export PDF/CSV

```
// Rules of Hooks: hooks ทั้งหมดต้องอยู่ก่อน early return
const [record, setRecord] = useState<CustomerRecord | null>(null)
const [fetching, setFetching] = useState(true)
const [sliderA, setSliderA] = useState(4)
// ... useMemo ...

// Early return หลัง hooks ทั้งหมด
if (loading || fetching) return <LoadingUI />
if (!isAdmin) return <ForbiddenUI />
if (error || !record) return <ErrorUI />
```

17. dataLayer

`src/engine/dataLayer.ts`

```
// Mock data — ใช้ระหว่าง development
// USE MOCK=true ทำให้ใช้ราคาจำลองแทน API จริง

export const THAI_TICKERS = ['PTT', 'KBANK', 'AOT', 'SCB', 'CPALL', 'ADVANC']
```

```

export const FOREIGN_TICKERS = ['AAPL', 'MSFT', 'NVDA', 'VTI', 'QQQ']
export const BOND_TICKERS = ['AGG', 'BND', 'TLT']
export const GOLD_TICKERS = ['GLD', 'IAU', 'GOLD']

const MOCK_PRICES: Record<string, number> = {
  PTT: 35.50, KBANK: 142.00, AOT: 68.25, AAPL: 189.50,
  MSFT: 415.20, VTI: 238.00, GLD: 195.30, AGG: 97.80,
}

const MOCK_FX = { USDTHB: 35.50 }

export async function getPrice(ticker: string): Promise<number | null> {
  if (USE MOCK) return MOCK_PRICES[ticker] ?? null
  // TODO: ค่า API จริง
}

export async function getFX(pair: string): Promise<number | null> {
  if (USE MOCK) return MOCK_FX[pair] ?? null
}

```

18. Config

package.json

```

{
  "name": "finance-advisor",
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.107.0",
    "html2canvas": "^1.4.1",
    "jspdf": "^4.2.1",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.23.1",
    "recharts": "^2.12.7"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.0",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "typescript": "^5.4.5",
    "vite": "^5.2.11"
  }
}

```

Library หลัก:

Package	ใช้ทำอะไร
react + react-dom	UI framework
react-router-dom	routing URL
recharts	กราฟ (pie, line chart)
@supabase/supabase-js	Database cloud
html2canvas	แปลง DOM เป็น image
jspdf	สร้างไฟล์ PDF
tailwindcss	CSS utility classes
typescript	Type checking
vite	Build tool

สรุปภาพรวมการทำงาน

```
ผู้ใช้เปิดเว็บ
↓
Login (ถ้าตั้งค่า Supabase) → Consent PDPA
↓
Questionnaire (11 ขั้นตอน, ~50 คำถาม)
  ↓ handleSubmit()
  computeMetrics() → computeRiskCoefficient() → assessAllRisks()
    ↓ บันทึกลง DB + เก็บใน AppStore
Output1: ตัวชี้วัด + การวัดความเสี่ยง
↓
Output2: broadAllocation(A) → PieChart + สัดส่วน
↓
Output3: efficientFrontier + tangency + tax → interactive chart
↓
Export CSV/PDF หรือดูประวัติ
```

หมายเหตุ: ไฟล์นี้ดึงโค้ดจริงจากทุกไฟล์ในโปรเจกต์ ข้อมูลตัวเลขทั้งหมด (เช่น threshold, น้ำหนัก, อัตราภาษี) มาจากโค้ดจริง ไม่ใช้การประมาณ